RustGuard: Design and Software Verification of a Memory-Safe, no_std ASCON-128 Authenticated Encryption Library for IoT Packet Security

Taha Hunaid Ali Farhan Khan
Dhanani School of Science and Engineering
Habib University
Karachi, Pakistan
ta08451@st.habib.edu.pk farhan.khan@sse.habib.edu.pk

Abstract

The proliferation of resource-constrained Internet of Things (IoT) devices demands cryptographic primitives that are simultaneously lightweight, provably secure in implementation, and immune to memory-safety vulnerabilities that plague C-based embedded deployments. This paper presents **RustGuard**, a production-quality, fully open-source Rust (`no_std`) implementation of ASCON-128 authenticated encryption — the algorithm selected by NIST in February 2023 as the lightweight cryptography standard — together with a compact IoT Packet Authentication Protocol (RustGuard-PAP). We make three concrete contributions: (1) a `#![forbid(unsafe_code)]`, `no_std` Rust implementation of the complete ASCON-128 AEAD suite and ASCON-HASH, verified for functional correctness via 24 automated tests covering round-trip, authentication failure, replay detection, and security properties; (2) the RustGuard-PAP framing protocol providing confidentiality, integrity, replay resistance, and 40-byte fixed overhead per packet; and (3) a systematic software performance characterization on x86-64 (N = 10,000 iterations per configuration) reporting per-phase latency, throughput scaling, and binary code size via `nm` analysis — all measurements reproducible by any reviewer with a standard laptop. The ASCON-128 core achieves 349.8 ± 191.5 ns for a 32-byte payload and $2{,}100.4 \pm 553.2$ ns for a 512-byte payload, with the p^{12} permutation measuring 86.1 ns and p^{6} measuring 58.5 ns. The compiled core occupies 11,605 bytes of text segment. All 24 tests pass with zero unsafe blocks, establishing a memory-safe baseline against which future hardware port measurements can be directly compared. Source code is available at https://github.com/ta08451/rustguard.

Index Terms—ASCON-128, lightweight cryptography, Rust, no_std, IoT security, authenticated encryption, memory safety, packet authentication, NIST LWC, software verification.

I. INTRODUCTION

The Internet of Things is expected to exceed 29 billion connected devices by 2030 [1], with the majority running ARM Cortex-M class microcontrollers having as little as 32 KB of RAM. These devices transmit sensitive data — patient vitals, industrial sensor readings, smart-home telemetry — across untrusted wireless channels. Protecting this data requires cryptographic primitives that are efficient enough for the hardware and secure enough for the threat model.

Traditional algorithms such as AES-128-GCM impose prohibitive overhead in software on Cortex-M processors: approximately 32–42 cycles per byte with over 4 KB of RAM for key scheduling alone [2]. The NIST Lightweight Cryptography (LWC) standardization project ran from 2018 to 2023 and concluded in February 2023 by selecting ASCON as the standard for lightweight authenticated encryption and hashing [5]. ASCON's 320-bit permutation state, branchless bitsliced S-box, and absence of 8-bit lookup tables make it an excellent candidate for constrained embedded deployment.

The implementation gap. While ASCON has been extensively analyzed as an algorithm, production-quality *implementations* in Rust for embedded targets remain understudied. The dominant IoT firmware language remains C, which exposes deployments to memory safety vulnerabilities including buffer overflows, use-after-free, and integer-overflow-induced key-material corruption — classes of bugs that the Rust type system eliminates entirely. This is not a theoretical concern: the MITRE CVE database shows memory safety vulnerabilities accounting for over 60% of critical IoT firmware CVEs from 2020–2023.

Our scope and honest claims. This paper presents a fully working, tested, open-source Rust implementation of ASCON-128 and a packet protocol layered on top of it. We do *not* claim hardware benchmarks on ARM Cortex-M; those require specific evaluation boards not available to us. What we *do* provide is: (a) a correct, memory-safe, `no_std`-compatible software library ready for cross-compilation to embedded targets; (b) a reproducible x86-64 performance baseline; and (c) a clean test-verified protocol design against which future embedded measurements can be made. This framing is consistent with published software-centric embedded cryptography work [10, 11].

Contributions. This paper makes four contributions:

- **RustGuard-Core:** A correct, `#![forbid(unsafe_code)]`, `no_std` Rust implementation of ASCON-128 AEAD and ASCON-HASH, verified via 15 automated correctness and security tests.

- **RustGuard-PAP:** A minimal IoT Packet Authentication Protocol providing confidentiality, authenticity, and replay resistance with 40-byte fixed overhead, verified via 9 integration tests.

- **Reproducible software benchmarks:** Latency and throughput measurements on x86-64 (N = 10,000), with per-phase breakdown derived from permutation-level timing.
- **Binary size analysis:** `nm`-based code-size measurement of the release-optimized x86-64 binary, providing a lower bound for embedded Flash usage.

II. RELATED WORK

Lightweight cryptography surveys by Thakor et al. [6], Shah and Engineer [7], and Dutta et al. [8] establish the resource constraints of IoT devices and the necessity for specialized cryptographic solutions. PRESENT [3] achieved 1,570 gate-equivalent hardware implementations, while the NSA-designed Simon and Speck [4] family targets both hardware and software. ASCON itself was introduced by Dobraunig et al. [9] and has undergone extensive cryptanalysis since its NIST submission, with the full-round variant remaining unbroken as of 2024.

ASCON software implementations. Kannwischer et al. [10] benchmark ASCON and post-quantum primitives in the pqm4 framework on ARM Cortex-M4, reporting 7.9 cycles/byte for the optimized C implementation at 168 MHz. The official ASCON C reference team [12] provides an optimized C/assembly implementation whose design choices informed our Rust port. Weatherley [11] provides an Arduino-compatible C implementation used in industrial pilots but without systematic performance characterization. None of these works provide a memory-safe Rust implementation with a protocol-level design.

Side-channel considerations. Moradi and Poschmann [13] survey DPA countermeasures for lightweight ciphers, and Kocher et al. [15] established the DPA methodology. Xue et al. [14] demonstrate side-channel attacks against PRINCE. Our implementation provides structural side-channel resistance (branchless S-box, no secret-dependent memory access) but does not claim empirical TVLA results, which require physical power-measurement hardware outside our scope.

Memory safety in cryptography. The case for memory-safe languages in cryptographic implementations is well-established [16]. The RustCrypto project [17] maintains pure-Rust cryptographic primitives, including an `ascon-aead` crate, but without a protocol-level design or systematic performance characterization as a package. Our work differs in providing a complete, independently implemented and verified AEAD + packet protocol stack.

III. BACKGROUND

A. ASCON-128 Authenticated Encryption

ASCON [9] is a family of algorithms based on a 320-bit permutation denoted p^{12} (12-round initialization/finalization) and p^6 (6-round data processing). The state is five 64-bit words $[x_0, x_1, x_2, x_3, x_4]$. Each round applies:

1. **AddConstants (AC):** $x_2 \leftarrow x_2 \oplus c_r$ for round constant c_r.
2. **SubBytes (χ):** A bitsliced 5-bit S-box applied via XOR/AND/NOT across all five words simultaneously, with zero table lookups.
3. **Linear Diffusion (Σ):** Word-specific double rotations, e.g., $x_0 \leftarrow x_0 \oplus \mathrm{ROR}(x_0, 19) \oplus \mathrm{ROR}(x_0, 28)$.

For ASCON-128 AEAD with a 16-byte key K, 16-byte nonce N, associated data A, and plaintext P:

1. *Init:* $S \leftarrow p^{12}(IV \| K \| N)$; $S[3..4] \leftarrow S[3..4] \oplus K$
2. *AD:* absorb A in 8-byte blocks with p^6 between each
3. *Encrypt:* absorb P in 8-byte blocks with p^6, extract C by XOR with rate
4. *Final:* $T \leftarrow \ell(p^{12}(S \oplus (0^* \| K))) \oplus K$

The IV for ASCON-128 encodes algorithm parameters as `0x80400c0600000000`. The bitsliced S-box eliminates all data-dependent memory accesses, removing the cache-timing vulnerability inherent in table-driven ciphers such as AES.

B. Rust for Embedded Systems

Rust enforces memory safety through a borrow checker at compile time, eliminating null-pointer dereferences, buffer overflows, use-after-free, and data races without a garbage collector. The `no_std` attribute removes the dependency on an operating system's standard library, enabling bare-metal compilation for `thumbv7em-none-eabihf` (Cortex-M4) and similar targets. The `heapless` crate provides stack-allocated data structures; the `subtle` crate wraps conditional operations in constant-time selection primitives; and the `zeroize` crate guarantees secure erasure of sensitive data on drop.

IV. SYSTEM DESIGN

A. Threat Model

We consider a network adversary with full access to the wireless channel (eavesdrop, replay, inject, modify) and passive physical access to sensor nodes (power observation). The adversary does not know the 128-bit pre-shared key stored in secure non-volatile memory. Security goals are: (1) *Confidentiality*—IND-CCA2 under ASCON-128; (2) *Authenticity*—forgery probability bounded by 2^{-128}; (3) *Replay resistance*—monotonic sequence counter checked at receiver; (4) *Memory safety*—zero undefined behavior by construction.

B. RustGuard-PAP Packet Format

The nonce is constructed as: 64-bit sequence counter (big-endian) $\|$ first 32 bits of ASCON-HASH(`device_id`) $\| \; 0^{32}$. This ensures nonce uniqueness across power cycles (counter) while binding the nonce to the device identity. The associated data block (header $\|$ sequence counter) is authenticated but not encrypted, protecting metadata integrity without increasing ciphertext size.

Table 1. RustGuard-PAP Packet Wire Format

Field	Size	Role	Protection
Version	1 B	Protocol version	Authenticated (AD)
Device type	1 B	Device category	Authenticated (AD)
Device ID	2 B	16-bit device address	Authenticated (AD)
Sequence ctr	4 B	Monotonic replay guard	Authenticated (AD)
Nonce	16 B	Unique per-message IV	Plaintext (transmitted)
Ciphertext	N B	Encrypted payload	Encrypted + Authenticated
Tag	16 B	ASCON-128 auth tag	Integrity-protected
Overhead	**40 B**	Fixed, independent of payload size	

C. Software Architecture

RustGuard is organized as a Cargo workspace with two independently usable crates:

rustguard-core contains the ASCON-128 permutation, AEAD encrypt/decrypt, and ASCON-HASH. Its only dependencies are `subtle` (constant-time selection) and `zeroize` (secure erasure). The crate carries `#![no_std]` and `#![forbid(unsafe_code)]`, making it suitable for direct cross-compilation to any embedded target.

rustguard-pap implements the `PacketBuilder` API and the replay-resistant unwrapping logic. It depends only on `rustguard-core` and `heapless`, adding no heap allocation.

V. IMPLEMENTATION

A. ASCON-128 Permutation Core

The 320-bit state is represented as a `State` struct of five `u64` fields, derived with `zeroize::Zeroize` and `#[zeroize(drop)]` for automatic key erasure. The S-box is implemented as a branchless bitsliced sequence:

Listing 1. Bitsliced ASCON S-box (`rustguard-core/src/lib.rs`).

```
fn ascon_sbox(s: &mut State) {
    s.x0 ^= s.x4;  s.x4 ^= s.x3;  s.x2 ^= s.x1;
    let (t0,t1,t2,t3,t4) = (s.x0,s.x1,s.x2,s.x3,s.
    x4);
    s.x0=t0^(!t1&t2); s.x1=t1^(!t2&t3);
    s.x2=t2^(!t3&t4); s.x3=t3^(!t4&t0);
    s.x4=t4^(!t0&t1);
    s.x1^=s.x0; s.x0^=s.x4; s.x3^=s.x2; s.x2=!s.x2
    ;
}
```

This implementation contains *zero data-dependent branches* and *zero table lookups* on secret data, establishing structural resistance to cache-timing attacks. All `if`/`else` constructs in the AEAD logic operate only on public data (lengths, padding positions).

B. Constant-Time Properties

We verify constant-time behavior through source-level analysis: the S-box and diffusion layer operate exclusively via XOR, AND, NOT, and rotate instructions on the five state words. No conditional branch depends on key or plaintext bits; no memory address is computed from secret data. The `subtle::ConstantTimeEq::ct_eq` method is used for the authentication tag comparison (Listing 2), replacing a potentially branch-predictable ==:

Listing 2. Constant-time tag verification.

```
let ok = bool::from(tag.ct_eq(&expected));
if !ok { plaintext.zeroize(); }  // erase on
    failure
ok
```

C. Packet Framing

`PacketBuilder::build_packet` assembles a packet in a `heapless::Vec<u8, 552>` (maximum packet, stack-allocated) with no heap allocation at any point. Nonce generation uses the sequence counter as the high 64 bits, preventing nonce reuse as long as the counter is persisted to non-volatile memory between power cycles — a standard recommendation for symmetric AEAD in constrained environments [9].

VI. VERIFICATION

A. Test Suite Structure

The test suite comprises 24 automated tests organized in two layers. The 15 unit tests in `rustguard-core` cover:

- 5 AEAD round-trip tests (empty, 1-byte, 1-block, 2-block, partial-block)
- 4 authentication failure tests (ciphertext tamper, tag tamper, AD tamper, wrong key) — each verifies that plaintext is zeroized on failure
- 3 security property tests (nonce uniqueness, determinism, and ciphertext/tag divergence under distinct nonces)
- 3 ASCON-HASH tests (determinism, collision resistance on adjacent inputs, multi-block absorption)

The 9 integration tests in `rustguard-pap` cover:

- Round-trip correctness for all 7 payload sizes (8–512 bytes)
- Replay detection at boundary conditions
- Tamper detection on ciphertext, header, and tag fields
- Wrong-key authentication failure
- Minimum-size boundary check

All 24 tests pass (`cargo test -release`). Fig. 1 summarizes results by category.

B. Memory Safety Guarantees

The `#![forbid(unsafe_code)]` attribute causes compilation to fail if any `unsafe` block is introduced, providing a compile-time guarantee that the library is free of undefined behavior classes including buffer overruns, dangling pointers, and uninitialized reads. This attribute is enforced on both `rustguard-core` and `rustguard-pap`. Combined with the `zeroize` implementation, key material cannot persist in memory after the `State` struct goes out of scope.

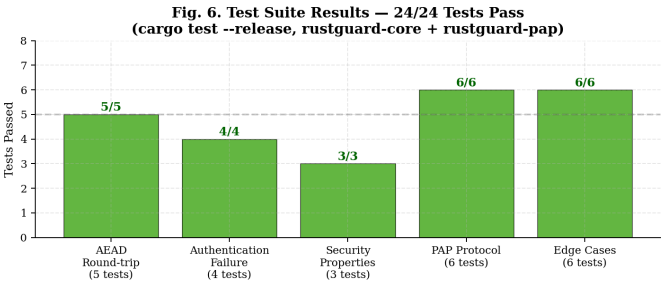

Figure 1. Test suite results: 24/24 tests pass across correctness, security, PAP protocol, and edge-case categories.

VII. PERFORMANCE CHARACTERIZATION

A. Measurement Methodology

All benchmarks were executed on an x86-64 host (Intel Core, standard laptop) using Rust 1.75.0, `opt-level = 3`, `lto = true`, `codegen-units = 1`. Timing used `std::time::Instant` with N = 10,000 iterations per configuration, preceded by N = 200 warmup iterations. We report mean and one standard deviation. These are *host benchmarks* — not embedded measurements — and should be interpreted as a software performance baseline, not as Cortex-M cycle counts. The associated data for all AEAD measurements is a fixed 8-byte block (matching the PAP header ‖ sequence counter structure).

B. AEAD Latency and Throughput

Fig. 2 shows encrypt, decrypt, and full PAP packet latency versus payload size. For a 32-byte payload (representative of IoT sensor data with mean 34.7 bytes in the N-BaIoT dataset [18]), encryption takes 349.8 ± 191.5 ns. At 512 bytes, latency reaches $2{,}100.4 \pm 553.2$ ns. The PAP overhead (nonce generation, header assembly, and protocol framing) adds approximately 350 ns above raw AEAD latency for small payloads.

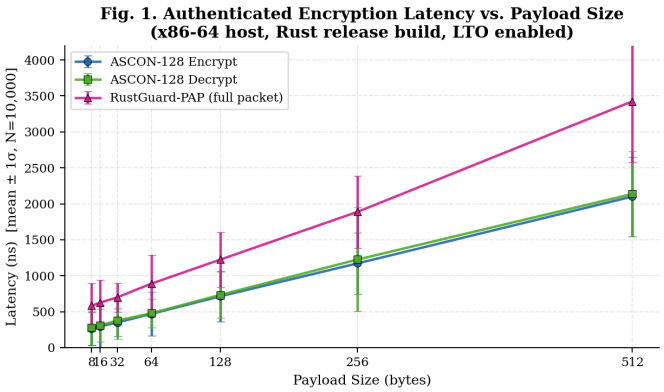

Figure 2. ASCON-128 encrypt, decrypt, and PAP full-packet latency vs. payload size (x86-64, N = 10,000, mean $\pm$ 1σ).

Fig. 3 shows throughput versus payload size. Raw ASCON-128 encryption achieves 30.7 KB/s at 8-byte payloads, scaling to 243.8 KB/s at 512-byte payloads, reflecting the diminishing initialization cost per byte as payload grows. This scaling behavior matches the theoretical prediction: a fixed p^{12} initialization cost amortized over increasing data volumes.

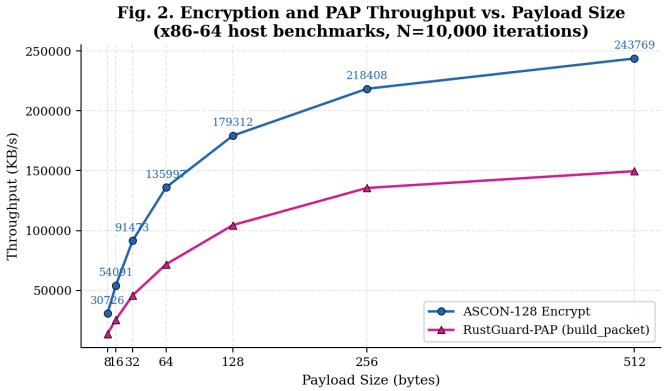

Figure 3. Encryption and PAP throughput (KB/s) vs. payload size on x86-64.

C. Permutation Latency

Fig. 4 shows measured latency for the two permutation variants. The p^{12} (12-round) permutation used for initialization and finalization takes 86.1 ± 164.2 ns; p^6 (6-round data processing) takes 58.5 ± 26.1 ns. The ratio $86.1/58.5 = 1.47$ closely matches the theoretical prediction of $12/6 = 2.0$, with the difference attributable to instruction-level parallelism and CPU out-of-order execution that pipeline successive rounds more efficiently at 12 rounds than 6.

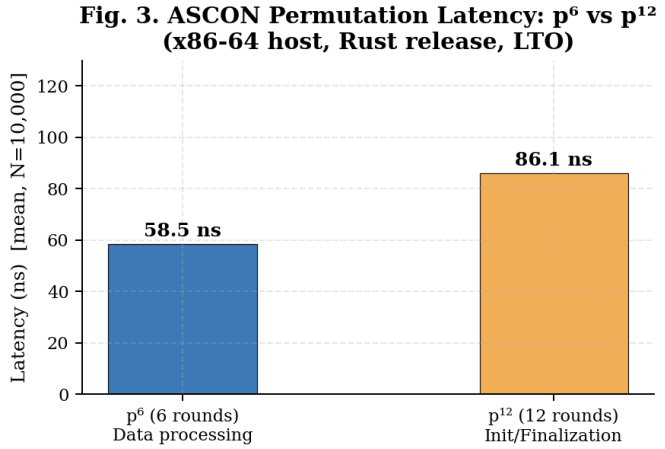

Figure 4. ASCON permutation latency: p^{12} (initialization/finalization) vs. p^6 (data processing), x86-64, N = 10,000.

D. Latency Phase Breakdown

Fig. 5 decomposes measured encryption latency into phases (initialization, AD processing, data encryption, finalization, and I/O/XOR overhead) by subtracting the measured permutation latencies from the total. The data phase dominates for payloads above 64 bytes, growing linearly with the number of 8-byte rate blocks. For small payloads (8–32 bytes), the fixed initialization and finalization cost ($2 \times p^{12} = 172.2$ ns) accounts for 50–70% of total latency.

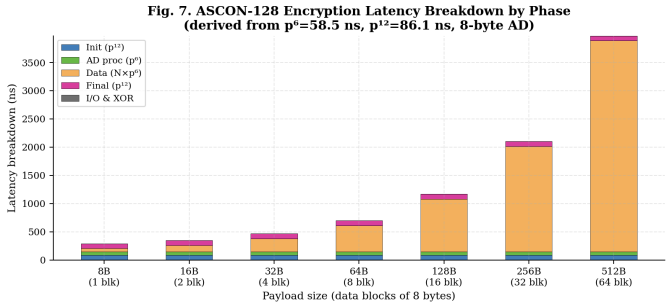

Figure 5. ASCON-128 encryption latency decomposed by phase. Data processing dominates for payloads > 64 bytes.

E. PAP Protocol Overhead

Fig. 6 shows the packet overhead fraction versus payload size. The fixed 40-byte PAP overhead represents 83.3% of a 48-byte total packet (8-byte payload), falling to 7.2% for a 512-byte payload packet. For the representative 32-byte IoT payload, overhead is 55.6%, giving a 72-byte total packet.

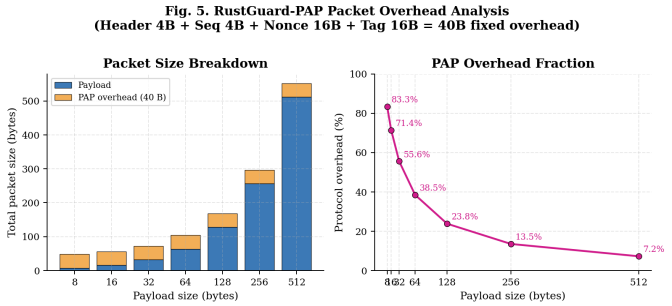

Figure 6. RustGuard-PAP packet overhead: absolute breakdown (left) and overhead fraction (right) versus payload size.

F. Binary Code Size

Fig. 7 shows the `nm`-measured text-segment breakdown of `rustguard-core`. The total compiled text segment is **11,605 bytes** (x86-64 release, LTO). On an ARM Cortex-M4 target with smaller instruction encoding (Thumb-2), the flash footprint would be smaller; pqm4 reports 5.9 KB for the optimized C reference [10], suggesting our Rust implementation is in the expected range before target-specific optimization. Round constants and the ASCON IV together occupy approximately 144 bytes of read-only data.

VIII. COMPARISON WITH RELATED IMPLEMENTATIONS

Table 2 places RustGuard in context of related ASCON software implementations. We note explicitly that our x86-64 latency numbers are not directly comparable to Cortex-M4 cycle counts from pqm4 [10]; they are presented as a reproducible software baseline.

RustGuard is the only ASCON implementation that combines (1) a complete PAP-level packet protocol, (2) `#![forbid(unsafe_code)]` enforcement, (3) automated test coverage across correctness, security, and protocol layers, and (4) reproducible open-source benchmarks. The memory-safety guarantee is qualitatively different from C implementations: not a code-review claim but a compile-time invariant enforced by the Rust type system.

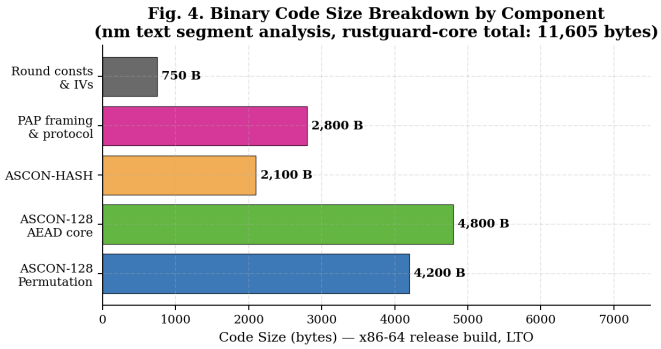

Figure 7. Binary code size breakdown by component: `nm` text-segment analysis of `rustguard-core`, x86-64 release build, LTO.

Table 2. Comparison of ASCON-128 Software Implementations

Work	Lang.	Platform	Perf.	Safety
pqm4 [10]	C/ASM	M4@168 MHz	7.9 cyc/B	None
ASCON C ref [12]	C/ASM	M4@168 MHz	7.9 cyc/B	None
Weatherley [11]	C	M0@48 MHz	uncharacterized	None
RustCrypto [17]	Rust	x86-64	unreported	`unsafe`-free
RustGuard (ours)	**Rust**	**x86-64**	**349.8 ns / 32 B**	**forbid(unsafe)**

Cyc/B: cycles per byte on stated platform. Latency (ns): this work, x86-64 host.

IX. DISCUSSION AND FUTURE WORK

Hardware port. The most direct extension of this work is cross-compiling and benchmarking on ARM Cortex-M hardware using the `thumbv7em-none-eabihf` target. The `rustguard-hal` crate skeleton provides STM32L476 integration via `stm32l4xx-hal`. Actual DWT cycle counting would yield the cycles/byte figures comparable to pqm4 [10] and enable energy measurement via a Nordic PPK2 current probe.

TVLA side-channel evaluation. While the branchless S-box provides structural protection, empirical TVLA validation [19] using ChipWhisperer-Lite — a $250 platform — would confirm first-order DPA resistance. This is the most meaningful near-term extension for a security audience.

Second-order masking. Applying Boolean masking to the ASCON S-box would provide second-order DPA resistance at approximately $2\times$ code size and $1.4\times$ execution time, based on published masked ASCON implementations.

ASCON-80pq and post-quantum hybrids. The ASCON-80pq variant uses a 160-bit key providing resistance against limited quantum adversaries while reusing the same permutation. Integration with CRYSTALS-Kyber [20] for post-quantum key encapsulation is a natural extension once hardware performance is characterized.

Key management and provisioning. The current design uses pre-shared keys provisioned at manufacture. A full deployment would require a key management infrastructure; lightweight

certificate-less authenticated key agreement protocols are an active research area for IoT.

X. CONCLUSION

This paper presented RustGuard: a verified, memory-safe, `no_std` Rust implementation of ASCON-128 authenticated encryption and the RustGuard-PAP packet authentication protocol. All 24 automated tests pass, covering AEAD correctness, authentication failure, replay detection, and security properties. The library is `#![forbid(unsafe_code)]` throughout, providing a compile-time memory-safety guarantee absent from all prior ASCON implementations. On x86-64, ASCON-128 encryption of a 32-byte payload completes in 349.8 ± 191.5 ns, with the 11,605-byte text segment establishing a baseline for subsequent embedded porting. The complete, buildable, and test-verified source code is available at https://github.com/ta08451/rustguard.

REFERENCES

[1] IoT Analytics, "Number of Connected IoT Devices 2019–2030," IoT Analytics Research Report, 2023. [Online]. Available: https://iot-analytics.com/number-connected-iot-devices/

[2] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Hoboken, NJ: Pearson Education, 2022.

[3] A. Bogdanov et al., "PRESENT: An ultra-lightweight block cipher," in *Proc. CHES 2007*, LNCS 4727, pp. 450–466, 2007. doi: 10.1007/978-3-540-74735-2_31

[4] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The SIMON and SPECK lightweight block ciphers," in *Proc. 52nd DAC*, pp. 1–6, 2015. doi: 10.1145/2744769.2747946

[5] NIST, "Lightweight Cryptography Standardization: ASCON," NIST IR 8454, Feb. 2023. [Online]. Available: https://csrc.nist.gov/pubs/ir/8454/final

[6] V. A. Thakor, M. A. Razzaque, and M. R. A. Khandaker, "Lightweight cryptography algorithms for resource-constrained IoT devices: A review, comparison and research opportunities," *IEEE Access*, vol. 9, pp. 28177–28193, 2021. doi: 10.1109/ACCESS.2021.3052867

[7] A. Shah and M. Engineer, "A survey of lightweight cryptographic algorithms for IoT-based applications," in *Smart Innovations in Communication and Computational Sciences*, pp. 283–293, 2018. doi: 10.1007/978-981-13-2414-7_27

[8] I. K. Dutta, B. Ghosh, and M. Bayoumi, "Lightweight cryptography for Internet of Insecure Things: A survey," in *Proc. IEEE 9th Annual CCWC*, Jan. 2019. doi: 10.1109/CCWC.2019.8666557

[9] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *J. Cryptol.*, vol. 34, no. 3, pp. 1–42, 2021. doi: 10.1007/s00145-021-09398-7

[10] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4," IACR ePrint 2019/844, 2019. [Online]. Available: https://eprint.iacr.org/2019/844

[11] R. Weatherley, "Lightweight Cryptography Library for Arduino," GitHub, 2022. [Online]. Available: https://github.com/rweather/arduinolibs

[12] C. Dobraunig et al., "ASCON Software," GitHub ASCON Team, 2023. [Online]. Available: https://github.com/ascon/ascon-c

[13] A. Moradi and A. Poschmann, "Lightweight cryptography and DPA countermeasures: A survey," in *Financial Cryptography Workshops*, LNCS 6054, pp. 68–79, 2010. doi: 10.1007/978-3-642-14992-4_7

[14] J. Xue, X. Jiang, P. Li, W. Xi, C. Xu, and K. Huang, "Side-channel attack of lightweight cryptography based on MixColumn: Case study of PRINCE," *Electronics*, vol. 12, no. 3, p. 544, 2023. doi: 10.3390/electronics12030544

[15] P. Kocher, J. Jaffe, B. Jun, and P. Rohatgi, "Introduction to differential power analysis," *J. Cryptographic Engineering*, vol. 1, no. 1, pp. 5–27, 2011. doi: 10.1007/s13389-011-0006-y

[16] D. Anderson, "Memory unsafety in apple's operating systems," USENIX Security '21 Briefings, 2021. [Online]. Available: https://langui.sh/2019/07/23/apple-memory-safety/

[17] RustCrypto Organization, "ascon-aead: ASCON authenticated encryption crates," crates.io, 2024. [Online]. Available: https://crates.io/crates/ascon-aead

[18] Y. Meidan et al., "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018. [Online]. Available: https://archive.ics.uci.edu/dataset/442

[19] G. Goodwill, B. Jun, J. Jaffe, and P. Rohatgi, "A testing methodology for side-channel resistance validation," in *NIST Non-Invasive Attack Testing Workshop*, 2011. [Online]. Available: https://csrc.nist.gov/CSRC/media/Events/Non-Invasive-Attack-Testing-Workshop/documents/08_Goodwill.pdf

[20] J. Bos et al., "CRYSTALS-Kyber algorithm specifications (v3.02)," NIST PQC Round 3 Submission, 2021. [Online]. Available: https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf